

Permissionless event ticketing, native on Sui — where **Walrus** is the entire **content backend** the product literally cannot exist without.

◆ Walrus

◆ Seal

◆ Enoki zkLogin

◆ Sui Move

● ON SUI · GAS SPONSORED

— THE STATUS QUO IS BROKEN

Your ticketing platform holds your money,
keeps your sales, and owns your data

gatekept keeps your sales, and owns your data.

◆ ADMIT - SUI OVERFLOW 2026

To host an event today you stitch together **five tools that never talk**: a ticketer, a payment processor, a check-in app, an email blaster, and a spreadsheet.

WHERE

Sui testnet

CONTENT LAYER

Walrus

The ticketer is **custodial** (it holds your cash), **gatekept** (you apply for permission to sell), and **closed** (nothing programmable lives on top of an event).

IT TICKET

sui.hostit.events

STATUS

Working app

We lived this across 6 real events. So we rebuilt it.

5+

disconnected tools to run one event

INDUSTRY ESTIMATE

~20%

of revenue lost to fees, fraud & no-shows

INDUSTRY ESTIMATE

Theirs

your sales, money & attendee data

— frozen the day they decide to

deploy your WAY

One permissionless platform. *No gatekeeper, no custodian, no takedowns.*

- ~ **Any wallet hosts** — connect (or sign in with Google) and publish the same afternoon. No application.
- \$ **Payouts withdraw from on-chain escrow** — sell in SUI, USDC, any Coin<T>. A flat 3% fee, no custodian, no skim.
- 🍷 **Gasless checkout & check-in** — buyers never touch a faucet; sponsored server-side via Enoki.
- 🗳️ **Native prediction markets** + an **end-to-end-encrypted ticket-holder forum** + POAP collectibles.

Live on Sui testnet today: sui.hostit.events — a real working app, not slideware.



0K attendees. 0 flagship events. Already shipped — *on EVM.*

HostIt is a faithful Sui Move port of the HostIt EVM Diamond — **same brand**, **same crew**. That ~50K is **real EVM-side traction**; the Sui app is the port, rebuilt **Walrus-native**.

We're not guessing whether organizers want this. **We've run the events.**

“Hosting Web3Lagos on HostIt was a smooth experience — easy integrations, hands-on support, and great post-event data.”



Awosika Ayodeji · Founder, Web3bridge

[Web3Lagos](#)

[Borderless](#)

[Anambra Web3](#)

[Blockchain @ Unilag](#)

[ProdFest](#)

[Fuel Africa](#)

Gasless

tickets & check-in, sponsored on Sui

Escrow

every payout backed on-chain



— THE QUESTION EVERY WALRUS DECK MUST ANSWER

Anyone can put one image on Walrus and call it *“decentralized.”*

That's the bar most submissions clear — and stop at. One blob, one screenshot, one press release. We think that misreads what Walrus is **for**.

Walrus is a content backend, not a demo prop. So we asked a harder question:

What would a real consumer app look like if Walrus held *everything* users actually read?

— THE THESIS

Walrus is the entire content backend. *There is no content database. Anywhere.*

Public metadata, media, **and** Seal-encrypted private data — all on Walrus, across **eight distinct use cases**, on a single **49-line SDK-free HTTP client**. No Postgres, no S3, no websocket holding content behind it.

SUI

the index & the truth — identity, ownership, logic, access policy. The on-chain event log is the only record of where content lives.

WALRUS

the store — every byte anyone actually reads: public metadata, media, art, reviews + encrypted forum, vault, drafts, AI memory.

SEAL

the lock — threshold encryption on the private subset, released only when a Move policy passes.

A HostIt event that doesn't live on Walrus *literally cannot exist.*

```
sources/event.move - fn create_event

// an Event is deliberately thin...
public struct Event has key {
  name, start_ms, end_ms, max_tickets, flags,
  uri: String,    // ← nothing but a Walrus blob id
}

// ...and the constructor refuses an empty uri:
assert!(string::length(&uri) > 0, E_EMPTY_URI);
// every Event MUST point at a Walrus blob
```

The mint aborts with `E_EMPTY_URI` if you try to create an event without a Walrus blob id.

Sui state is structurally incomplete without Walrus — a property you prove by reading one line and trying to violate it, not a claim you take on faith.

BREADTH FEW CAN MATCH
VOID WITHOUT WALRUS

Eight genuinely different uses. *One tiny client.*

01

PUBLIC

Event metadata

JSON — the canonical content of every event · `Event.uri`

02

PUBLIC

Cover images

raw bytes, straight from the aggregator as `<img src>`

03

PUBLIC

POAP artwork

blob id cloned into the NFT via Sui Display

05

PUBLIC

Public reviews

POAP-gated — only proven attendees can post

04

SEAL

Ticket-gated forum

one encrypted blob per message; non-holders can't read

06

SEAL

Self vault

encrypted to your own address — `seal_approve_self`

07

SEAL

Event drafts

full in-progress form · 30-epoch TTL · no backend

08

SEAL

AI organizer memory

TEE-relayed embeddings that ground AI drafting

Four public use cases — all through web/lib/walrus.ts. No content database anywhere.

PUT to write. *GET* to read. No SDK. That's it.

web/lib/walrus.ts — 49 lines, no SDK

```
// write — returns a content-addressed blob id
PUT {publisher}/v1/blobs?epochs=N

// read — anyone, anywhere
GET {aggregator}/v1/blobs/{id}

// dedup handled: newlyCreated ?? alreadyCertified
// blobUrl() → a cover image is just an <img src>
```

- T** **TTL is first-class** — 10 epochs default, 30 for drafts that must outlive a session.
- ⇄** **Dedup handled** — newlyCreated vs alreadyCertified resolved transparently.
- u** **Uploads cached** so a retry never re-stores the same bytes.
- 🛡** **Both origins pinned in the CSP** — graceful fallback to public testnet endpoints.

The entire content layer of a consumer app fits in **a file you can read over coffee.**

— WALRUS + SEAL · THE HEADLINE PAIRING

One encrypted blob. Two on-chain policies.

Access composes; content stays single-copy.

```
Seal.encrypt(message, event_id) → Walrus.store({id, ct}) → forum::post(blob_id) anchors it on-chain
```

decrypt the **same ciphertext** via either Move policy:

✓ `seal_approve_ticket` — a ticket holder **OR** ✓ `seal_approve_organizer` — the OrganizerCap

- ⌘ **Authorization composes on-chain** — two independent Move policies, **one blob**. Neither party re-encrypts.
- ⦿ **The aggregator only ever sees ciphertext.** Walrus holds exactly one copy; Seal holds the key; Sui decides who gets it.
- ⦿ **Namespacing is real** — organizer-only data is domain-separated, so a ticket-holder policy can never decrypt it.

— IMMUTABILITY, DONE RIGHT

We embrace Walrus's permanence *instead of fighting it.*

A post is an immutable event **plus** an immutable blob — so moderation is by **tombstone** (`PostModerated`, keyed on the blob id), never deletion.

No content database means **no takedowns, no silent edits, no platform that can rewrite your event's history.**

Permanence isn't a constraint we worked around. It's the property that makes the whole thing **trustworthy.**

Bet on whether it sells out — *settled trustlessly, no oracle.*

Every event opens native parimutuel markets: a **Sellout Clock** and a **final-tickets-sold range**. They settle by reading the contract's own `event::minted()` at expiry — anyone can settle; goalposts are snapshotted at creation so they can't move.

no *oracle*

no *keeper*

no *house*

no *fee*

NOTE

DeepBook Predict has no self-serve oracle on testnet — so we shipped our own that settles on event state. **Impossible on a Web2 ticketing platform.**



— ONBOARDING · GASLESS

The decentralization is *invisible* to the user.

Sign in with Google via **Enoki zkLogin** — no seed phrase, no wallet hurdle, no faucet. Checkout and check-in route through server-side sponsorship, scoped by one allowlist. Yet every byte read is on Walrus and every private message is Seal-encrypted.

— WHY THIS WINS THE WALRUS TRACK

Google sign-in · zkLogin

Gas sponsored

SUI · USDC · any Coin<T>

3% fee → escrow

Your keys, your tickets

Load-bearing, not decorative. *The contract enforces it.*

The hard crypto is fully hidden behind consumer-grade flow — so a real attendee at Web3Labs can actually use it.

- ① **Load-bearing** — the Move code asserts a non-empty Walrus uri. No Walrus, no event.
- ② **Breadth** — 8 distinct uses, public & encrypted, on one 49-line client.

③ **Composition** — Walrus + Seal gated by on-chain Move policy; same blob, two policies.

④ **No content database — anywhere.** Sui indexes, Walrus stores. Full stop.

⑤ **Real product** — ported from a platform that served ~50K attendees.

● ADMIT ONE